

Veratas: an inspectable RAG system with sentence-level citation checking

CSD358 Information Retrieval, mid-semester hackathon. Track **T1: Retrieval-Augmented Generation and trustworthy answers**

Team Rahul Vaidhya (2410110259), Akshat Bansal (2410110039), Ananmay Dubey (2410110513), Yug Gupta (2410110490)

Code https://github.com/nwopes/notebook_lm_clone (README: setup, run, data, status)

Demo video [\[unlisted YouTube / Drive link\]](#)

1. Problem and track relevance

User need. Students and teams increasingly ask LLM chat tools about their own documents (lecture notes, textbooks, reports). A plain LLM answers fluently, but the reader cannot tell which parts come from the sources and which are invented. Our users need (a) answers grounded only in the documents they uploaded, (b) a pointer from every claim to the exact passage and page, (c) a signal when a claim is *not* supported, and (d) a refusal when the sources don't contain the answer.

Why it is a T1 problem. The track's thesis is that "an LLM is only as good as what gets retrieved for it". Retrieval-augmented generation (Lewis et al., 2020) moved the quality problem from the generator to the retriever, and recent work studies whether generated citations actually support the generated text (Gao et al., 2023, ALCE; Liu et al., 2023, verifiability of generative search engines: only about half of sentences in commercial answers were fully supported by their citations). We therefore built the retriever as a component the user can open up and inspect: postings, df/idf, per-term score contributions, fusion ranks. We added a **citation checker** that re-scores every generated sentence against the chunk it cites. That is the second sample idea for this track, extended with inherited citations, sentence windows and a measured evaluation.

Papers referred: Lewis et al. (RAG); Robertson & Zaragoza (BM25); Cormack et al. (RRF, SIGIR'09); Thakur et al. (BEIR); Wadden et al. (SciFact); Gao et al. (ALCE); Liu et al. (verifiability); Manning, Raghavan & Schütze (IIR textbook). See References.

2. How we used IR

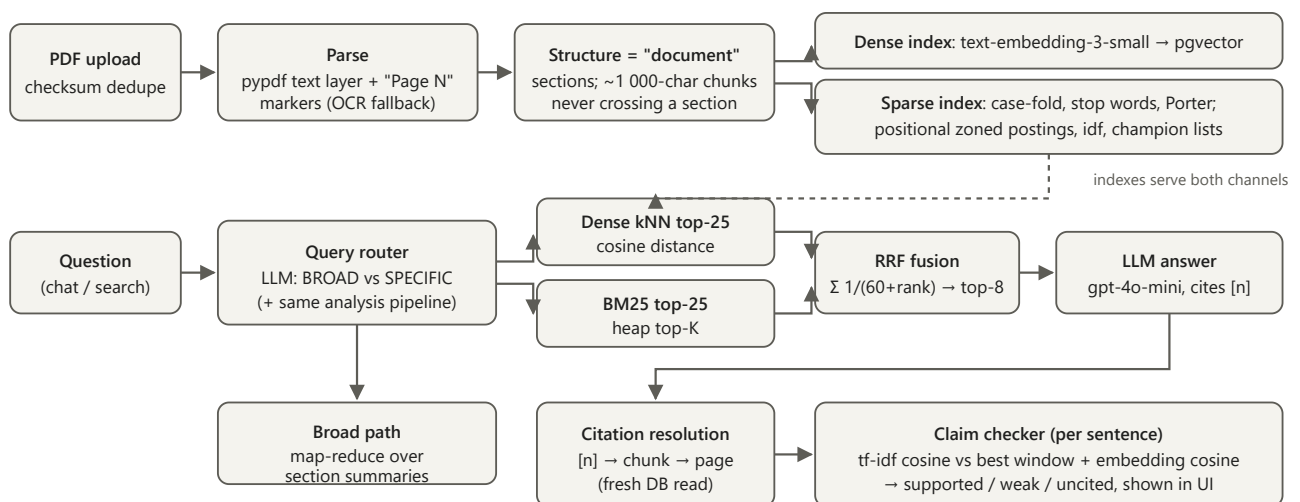


Figure 1. Pipeline. Top row: ingestion (background worker). Middle: specific-question path. Bottom: broad-question path and answer verification. The Search page exposes the sparse index directly (ranked, Boolean, phrase) with full traces.

IR principle (lecture topic)	Where in the code	Why / how it is used
Deciding what a document is	<code>services/ingestion/structuring.py</code> <code>_build_sections_and_chunks</code>	The retrieval unit is a ~1 000-char chunk inside one leaf section (one page or one recovered heading). Small enough to cite precisely, large enough to hold a definition. Every chunk keeps char offsets and a page.
Tokenization, normalization, case folding, stop words, Porter stemming	<code>retrieval/sparse/text.py</code> <code>tokenize</code> , <code>analyze_with_positions</code>	NFKC + <code>casefold</code> , a short function-word stop list (positions are kept so phrase distance stays correct), NLTK Porter (original algorithm). The same pipeline runs on documents and queries; the Search page shows every stage.
Inverted index: dictionary + positional postings; zone index	<code>retrieval/sparse/index.py</code> <code>InvertedIndex.build</code>	Postings per (zone, term) sorted by doc id with positions; zones = heading / body. df counts a document once across zones. Built per notebook and cached (<code>sparse_channel.py</code>).
Boolean retrieval, postings intersection, query optimization	<code>sparse/scoring.py</code> <code>intersect</code> , <code>union</code> , <code>and_not</code> , <code>_intersect_many</code> ; parser + traces in <code>sparse/trace.py</code>	Linear merge of sorted postings; AND terms processed in increasing-df order (shown to the user as the merge order). Malformed queries get a 422 with a reason.
Positional index and phrase queries	<code>scoring.py</code> <code>phrase_search</code>	Exact phrases such as "Ekman transport": intersect, then check consecutive positions. Snippets are centred on the real occurrence.
tf-idf, VSM, cosine, SMART Inc.ltc	<code>scoring.py</code> <code>search(scheme="tfidf")</code> ; norms precomputed in <code>index.py</code> <code>doc_norm</code>	Log-tf documents with cosine normalization, ltc queries; the score is the dot product. Each hit lists every term's tf, idf and weight ("Why this score?").
BM25 (beyond syllabus)	<code>scoring.py</code> <code>search(scheme="bm25")</code> , $k_1=1.2$, $b=0.75$	The production lexical channel (<code>SPARSE_RETRIEVAL_MODE=bm25</code>). It wins over tf-idf on both evaluations (Section 5).
Efficient scoring: term-at-a-time, heap top-K, index elimination, champion lists	<code>scoring.py</code> <code>search</code> , <code>_champion_candidates</code> ; <code>index.py</code> <code>champions (r=50)</code>	Accumulators + <code>heapq.nlargest</code> ($O(N \log K)$); an idf threshold drops near-stop-word terms; champion lists cut SciFact query time $2.4\times$ ($3.08 \rightarrow 1.26$ ms) with identical nDCG@10.
Zone weighting	<code>search(zone_weights=...)</code> , <code>SPARSE_HEADING_ZONE_WEIGHT=2.0</code>	Score = $\sum_z w_z \cdot \text{score}_z$. Measured honestly: the heading boost <i>hurt</i> SciFact (nDCG $0.686 \rightarrow 0.655$) and over-weights short headings for tf-idf (Section 5.2).
Dense retrieval + rank fusion (beyond syllabus)	<code>retrieval/service.py</code> ; <code>retrieval/fusion.py</code> <code>fuse_rrf</code>	pgvector cosine kNN and BM25 each return 25 candidates; Reciprocal Rank Fusion ($k=60$) merges them and the top 8 become numbered context blocks.
Cosine similarity for verification	<code>chat/citation_check.py</code> <code>score_claims</code> , <code>chunk_windows</code> ; <code>scoring.py</code> <code>tfidf_cosine</code>	Each answer sentence is an ltc vector; lexical = max cosine against 1–2-sentence windows of each cited chunk (idf from the notebook index); semantic = embedding cosine; combined = mean, with threshold 0.33.
Precision / recall / P@k, MRR, nDCG	<code>eval/metrics.py</code> , <code>eval/run_ablation.py</code> , <code>eval/textbook_eval.py</code>	Retriever evaluated separately from the generator, on a public benchmark and on our own judged queries.

3. Beyond IR

- **LLM generation with a grounding contract** (gpt-4o-mini via OpenRouter): answer only from the numbered blocks, cite as [n], otherwise reply with a fixed refusal sentence. [n] markers are mapped back

to chunks by a fresh database read, so a citation can never point at text that was not retrieved.

- **Query routing:** one cheap LLM call classifies the question as BROAD or SPECIFIC. Broad questions ("summarize these chapters") cannot be answered from top-8 chunks, so they go to a map-reduce over LLM-written **section summaries** produced at ingestion (enrichment). We tightened the classifier prompt after finding "What is X?" misrouted as broad; 9/9 test questions now route correctly.
- **Semantic outline:** an LLM recovers real headings (e.g. "1.3.2 Oceanic circulation") from the page text, which populates the heading zone.
- **Systems:** FastAPI, Postgres + pgvector, Redis/arq background ingestion, React UI with streaming (SSE), multi-tenant orgs, notebook sharing and folder-level access roles. 495 backend and 224 frontend automated tests.

4. Novelty and creativity

1. **A glass-box retriever inside a production-style RAG app.** NotebookLM-like tools hide retrieval. Our Search page and debug panel show the analyzer stages, postings merge order, df/idf, per-term/per-zone contributions and fusion ranks for the exact chunks the LLM saw. The same from-scratch index is the live lexical channel of the hybrid retriever, not a separate toy.
2. **Sentence-level citation checking with inherited citations.** LLMs often put one [n] at the end of a paragraph. We propagate the marker to the paragraph's earlier sentences, score each sentence against its *best-matching window* of the cited chunk (not the whole chunk), and fuse lexical and semantic evidence. The UI labels each sentence supported / weak / uncited with its scores.
3. **We measured what the checker cannot do.** On SciFact we separate easy negatives (random abstract), hard negatives (on-topic, BM25-mined) and contradictions. The checker almost never accepts an off-topic citation, but it cannot detect contradiction. We report this rather than hide it (Section 5.3).
4. **Parser fidelity as a retrieval problem.** While testing we found that the hosted PDF parser silently *dropped bold and italic spans*. In a textbook those are the defined terms, so "what is Ekman transport" was refused even though the book defines it. Switching to the PDF's own text layer (keeping per-page markers and real page numbers) raised MRR@10 for **every** retrieval method (Fig. 2). Most RAG evaluations assume parsing is lossless; we measured that it was not.

5. Evaluation

5.1 Retriever ablation on BEIR SciFact (public judgments, 5 183 abstracts, 300 test queries)

method	P@1	P@5	R@10	MRR@10	nDCG@10	ms/q
tf-idf Inc.Itc (baseline)	.483	.147	.759	.579	.619	3.10
BM25	.550	.166	.832	.647	.686	3.08
BM25 + title zone ×2	.520	.159	.796	.617	.655	2.94
BM25 + champion lists	.550	.166	.832	.647	.686	1.26
dense (text-embedding-3-small)	.593	.178	.854	.680	.717	1.22
hybrid: BM25 + dense, RRF	.610	.178	.866	.700	.735	4.42

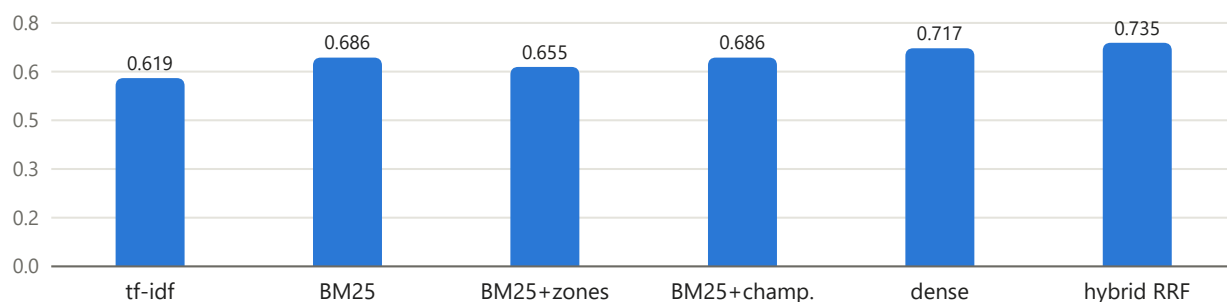


Figure 3. SciFact nDCG@10 (y-axis 0–0.8). Rerun on 2026-10-07; sparse rows reproduce the committed results exactly, dense differs in the 4th decimal.

BM25 beats the tf-idf baseline on every metric (+0.067 nDCG@10). Dense beats BM25, and RRF fusion is best: per query, BM25 wins 52 and dense wins 66 of 300 (margin > 0.1), so the two channels fail on different queries, which is why fusion helps. BM25 wins on queries with rare exact terms (gene names such as "CTCF"); dense wins on paraphrased claims. Champion lists give the same quality 2.4× faster.

5.2 Our own judged queries on the demo corpus (climate textbook, 2 chapters, 57 pages)

We wrote 18 questions (12 using the book's key term, 5 paraphrased without it, 1 misspelled) and judged relevance at the **page** level by reading the relevant sections, then re-checked every query against all 57 pages (six judgments corrected, e.g. a reference-list page removed) (`eval/textbook_qrels.json`). A retrieved chunk is relevant if its page span covers a judged page. Everything runs against the live app (`eval/textbook_eval.py`), so parsing, chunking and ranking are all measured together.

method (after parser fix)	P@5	P@10	R@10	Hit@5	MRR@10	MRR@10 by query kind	n	tf-idf	BM25	hybrid	dense
tf-idf Inc.Itc (baseline)	.578	.439	.961	.833	.854	keyword	12	.927	1.00	1.00	.903
BM25	.578	.400	.950	.944	.909	paraphrase	5	.829	.850	.900	1.00
hybrid RRF (production)	.633	.450	.961	1.00	.935	misspelled ("ekamn")	1	.111	.111	.333	1.00
dense only	.600	.450	.970	1.00	.935						

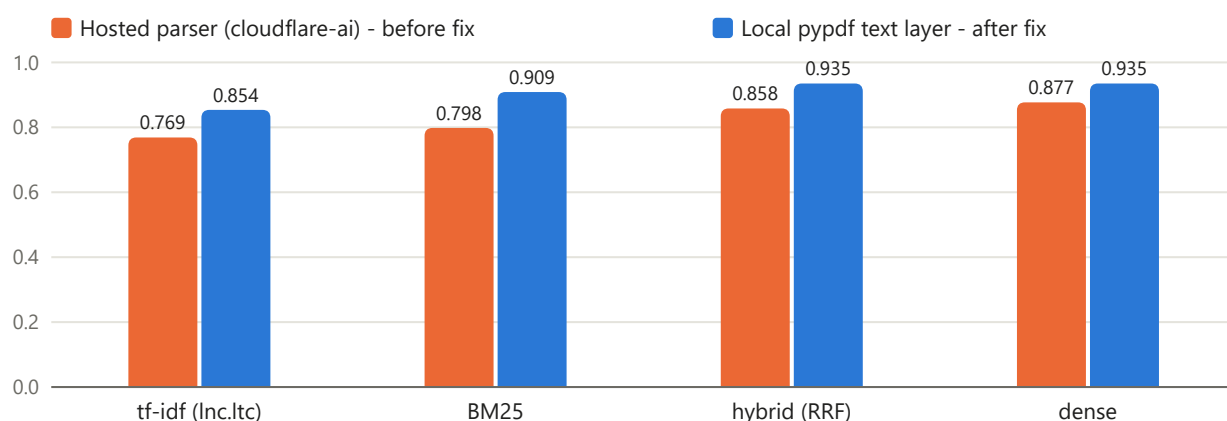


Figure 2. Same 18 queries, same retrieval code, two parsers. The hosted parser lost emphasized terms (e.g. the 3 occurrences of "Ekman" in Ch. 1 became 1, and the sentence defining it lost the term), which hurt every method; the local text layer fixes it.

Reading the results. Hybrid is the best production choice: it has the highest P@5 and P@10, ties dense on MRR@10, and finds a relevant page in the top 5 for *every* query. On keyword queries BM25 ranks a relevant page first every time (MRR 1.00) while dense is worse (0.903, e.g. q04 "thermohaline circulation": dense ranks it 3rd, BM25 1st). On paraphrases and the typo dense is clearly better, and fusion partly recovers it. tf-idf Inc.Itc

loses to BM25 for a concrete reason we could trace with the contribution view: in "What is Ekman transport?", the heading-zone match on "transport" in short headings such as "2.1.5.2 Heat transport" outweighs the rare body term "ekman" after cosine normalization (MRR 0.12 vs 1.00). BM25 avoids it: it normalizes each zone by its length relative to the average zone length and saturates tf, whereas cosine normalization inflates every term of a two-word heading. Caveat: with only 18 queries and 1–5 relevant pages each, R@10 saturates; differences of a few hundredths are not significant.

5.3 Citation checker on SciFact claims (does a "supported" label mean anything?)

pair kind (dev)	n	lexical	semantic	combined	% "supported" @0.33
claim + its SUPPORT abstract	138	.497	.651	.574	97.8%
claim + CONTRADICT abstract	71	.412	.618	.515	88.7%
claim + on-topic wrong abstract (BM25 hard negative)	300	.359	.494	.426	80.7%
claim + random abstract	300	.019	.209	.114	1.0%

Against off-topic citations the checker is near perfect (ROC-AUC 0.999, $P = R = 0.978$ at 0.33). Against on-topic wrong citations it still ranks well (AUC 0.808) but the threshold is too lenient: tuning on the train split gives 0.49 (dev F1 0.524 → 0.609). The best-matching window points at a gold rationale sentence 55% of the time (random window: 27%, lead sentence: 6.5%). It is **not** an entailment test: SUPPORT vs CONTRADICT AUC is 0.61.

5.4 Working-system sanity check

A scripted end-to-end run against the live stack (fresh account, 3 PDFs, real models) passes 29 of 30 scripted checks; the one miss is our own over-strict check that tf-idf's top hit for "Ekman transport" mentions Ekman, which is exactly the zone-normalization effect analysed in 5.2. Verified: sign-up/login, upload and checksum dedupe, non-PDF rejection (415), READY in ~25 s, ranked/Boolean/phrase search ($|\text{ekman}| = |\text{ekman AND upwelling}| + |\text{ekman AND NOT upwelling}|$: $3 = 2 + 1$), malformed Boolean → 422, cited streaming answers with page numbers and claim checks (e.g. "what is ekamn transport" → pages 9–10, 3/3 claims supported), refusal for an off-corpus question ("Who won the 2022 FIFA World Cup?"), broad summary, history, trace, feedback, overview, and cross-org isolation (404).

6. Limitations and next steps

- **No entailment.** "Supported" means on-topic lexical/semantic support. 89% of contradicting claims pass. Next: an NLI cross-encoder on (sentence, best window), and moving the threshold to the tuned 0.49.
- **Sparse channel has no spelling tolerance** ("ekamn" matches only "transport", MRR 0.11). Next: k-gram index + edit distance spelling correction (IIR ch. 3), shown in the trace.
- **Small judged set** (18 queries, one corpus, judged by the team, single assessor). Next: 50+ queries, two assessors and agreement (κ).
- **Freshness gap:** section summaries arrive ~1 min after a document is READY; until then broad questions fall back to normal retrieval. Broad answers cite sections and have no claim checks.
- **Latency:** 5–15 s per answer (LLM-bound); broad questions add a classification and map-reduce calls.
- **Security items from QA** (e.g. notebook chat history visible to all members) are tracked in `.claude/known-issues.md`.

Course-project roadmap. (1) Spelling correction + query-term proximity scoring in the sparse channel; (2) NLI-based claim verification and calibrated thresholds; (3) learning-to-rank over BM25, dense, zone and proximity features using the feedback (thumbs) the app already stores; (4) larger judged query set with inter-assessor agreement.

7. Work division

Member	Components owned
Akshat Bansal	Core system: ingestion pipeline, from-scratch sparse IR core (analyzer, positional/zone index, tf-idf/BM25, Boolean/phrase), hybrid RRF retrieval, RAG chat and citation resolution
Rahul Vaidhya	Local deployment and end-to-end testing, parser-fidelity fix (pypdf text layer), query-router fix, streaming error handling, judged textbook evaluation, report and README
Ananmay Dubey	Citation checker and its SciFact evaluation (hard negatives, contradiction analysis, threshold tuning)
Yug Gupta	SciFact retrieval ablation (BEIR harness, sparse vs dense analysis), Search page traces, demo video

8. AI-use declaration

- **AI coding agent:** Claude Code (Anthropic) was used throughout development to write and refactor most of the backend and frontend code and the tests, under the team's review. The repository's `.claude/` folder records that workflow. In the final session it was also used to set up and debug the local environment, find and fix the PDF-parser defect and the query-router misclassification, return proper errors from the streaming endpoint, write `eval/textbook_eval.py`, and draft this report and the README.
- **Relevance judgments** for the 18 textbook queries were drafted with the AI agent from the book's section structure and passages, then re-checked against all 57 pages with the same agent (six judgments corrected, e.g. a bibliography page removed).
- **LLMs and models inside the system** (via OpenRouter): `gpt-4o-mini` for answers, query routing, section summaries, overview and heading recovery; `text-embedding-3-small` for dense retrieval and the semantic half of the checker. Ranking, the inverted index, Boolean/phrase processing and evaluation metrics are our own code. Libraries: NLTK (Porter stemmer only), pypdf (PDF text layer), pgvector (vector kNN), numpy (evaluation).

References

1. P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
2. S. Robertson, H. Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in IR, 2009.
3. G. V. Cormack, C. L. A. Clarke, S. Büttcher. Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods. SIGIR 2009.
4. N. Thakur et al. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. NeurIPS Datasets & Benchmarks 2021.
5. D. Wadden et al. Fact or Fiction: Verifying Scientific Claims (SciFact). EMNLP 2020.
6. T. Gao, H. Yen, J. Yu, D. Chen. Enabling Large Language Models to Generate Text with Citations (ALCE). EMNLP 2023.
7. N. F. Liu, T. Zhang, P. Liang. Evaluating Verifiability in Generative Search Engines. Findings of EMNLP 2023.
8. C. D. Manning, P. Raghavan, H. Schütze. Introduction to Information Retrieval. Cambridge University Press, 2008.
9. M. F. Porter. An algorithm for suffix stripping. Program 14(3), 1980.
10. Data: H. Goosse et al., Introduction to climate dynamics and climate modelling, climate.be/textbook (Ch. 1–2); SciFact via BEIR.